

Blockchain Experiment 2

Name: Atharva Lotankar

Class: D20C **R.No.:** 20

Date: 19 / 08 / 2026

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Program #1: Python program using Google Colab for Interactive Blockchain Logic & Demo Execution

Program:

```
import datetime
import hashlib
import json

class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
```

```

# Create Genesis Block (Block 1)
self.create_block(proof=1, previous_hash='0')

def create_block(self, proof, previous_hash):
    """
    Creates a new block in the blockchain and attaches current transactions.
    Clears pending transactions list afterward.
    """
    block = {
        'index': len(self.chain) + 1,
        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': self.transactions
    }
    self.chain.append(block)
    # Empty transaction list after mining
    self.transactions = []
    return block

def get_previous_block(self):
    return self.chain[-1]

def create_transaction(self, sender, receiver, amount):
    """
    Adds transaction to pending list and returns target block index.
    """
    self.transactions.append({
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    })
    return len(self.chain) + 1

def proof_of_work(self, previous_proof):
    """
    Cryptographic puzzle: searches for a golden nonce where
    SHA-256 output starts with '000' leading zeros.
    """
    new_proof = 1
    check_proof = False

    while check_proof is False:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()

        if hash_operation[:3] == '000':
            check_proof = True

```

```

        else:
            new_proof += 1

    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1

    while block_index < len(chain):
        block = chain[block_index]

        # 1. Verify previous hash matching
        if block['previous_hash'] != self.hash(previous_block):
            return False

        # 2. Verify Proof of Work target (3 leading zeros)
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()

        if hash_operation[:3] != '000':
            return False

        previous_block = block
        block_index += 1

    return True

# Instantiate Blockchain
blockchain = Blockchain()

def mine_block():
    # Mine ONLY if transaction list is not empty
    if not blockchain.transactions:
        return {'message': 'No transactions to mine. Add some transactions first!'}

    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(proof, previous_hash)

```

```

    return {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

def add_transaction(sender=None, receiver=None, amount=None):
    if sender is None:
        sender = input("Enter sender: ")
        receiver = input("Enter receiver: ")
        amount = float(input("Enter amount: "))

    index = blockchain.create_transaction(sender, receiver, amount)
    return {'message': f'Transaction will be added to Block {index}'}

def get_chain():
    return {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

def is_valid():
    valid = blockchain.is_chain_valid(blockchain.chain)
    if valid:
        return {'message': 'All good. The Blockchain is valid.'}
    return {'message': 'Houston, we have a problem. The Blockchain is not valid.'}

# Block 1 is already auto-generated as Genesis Block

# Add Transaction & Mine Block 2
print(add_transaction(sender="Atharva", receiver="Manu", amount=100.0))
print(json.dumps(mine_block(), indent=2))
print("-" * 50)

# Add Transaction & Mine Block 3
print(add_transaction(sender="Manu", receiver="Ronit", amount=45.5))
print(json.dumps(mine_block(), indent=2))
print("-" * 50)

# Add Transaction & Mine Block 4
print(add_transaction(sender="Ronit", receiver="Me", amount=12.0))
print(json.dumps(mine_block(), indent=2))
print("--- FULL BLOCKCHAIN DETAILS ---")
print(json.dumps(get_chain(), indent=2))

print("\n--- VALIDITY CHECK ---")
print(json.dumps(is_valid(), indent=2))

```

```

print("Functions Menu:")
print("=====")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
print("4. Add transaction to the blockchain")

try:
    choice = int(input("Enter your choice: "))
    if choice == 1:
        print(json.dumps(mine_block(), indent=2))
    elif choice == 2:
        print(json.dumps(get_chain(), indent=2))
    elif choice == 3:
        print(json.dumps(is_valid(), indent=2))
    elif choice == 4:
        print(json.dumps(add_transaction(), indent=2))
    else:
        print("Invalid choice")
except ValueError:
    print("Please enter a valid numeric choice.")

```

Output:

```

Functions Menu:
=====
1. Mine a block
2. Display the chain
3. Check the validity of the chain
4. Add transaction to the blockchain
Enter your choice: 2
{
  "chain": [
    {
      "index": 1,
      "timestamp": "2026-08-19 06:58:29.364931",
      "proof": 1,
      "previous_hash": "0",
      "transactions": []
    },
    {
      "index": 2,
      "timestamp": "2026-08-19 06:58:32.805839",
      "proof": 533,
      "previous_hash":
"c2103ac5877977c82bd3c3581d40f8b987acc9bda75099214d0e9eef53a41dc3",
      "transactions": [
        {
          "sender": "Atharva",
          "receiver": "Manu",
          "amount": 100.0
        }
      ]
    }
  ]
}

```

```

    },
    {
        "index": 3,
        "timestamp": "2026-08-19 06:58:32.818131",
        "proof": 11028,
        "previous_hash":
"7309039886650294b6ca39efea642fb8cb4c70503df9f92692e3a1b670a6d93f",
        "transactions": [
            {
                "sender": "Manu",
                "receiver": "Ronit",
                "amount": 45.5
            }
        ]
    },
    {
        "index": 4,
        "timestamp": "2026-08-19 06:58:32.818917",
        "proof": 396,
        "previous_hash":
"e75a3d7bca038a4f82229af1882c3febe4278c7da12cd48d7226720a779f5b93",
        "transactions": [
            {
                "sender": "Ronit",
                "receiver": "Me",
                "amount": 12.0
            }
        ]
    },
    {
        "index": 5,
        "timestamp": "2026-08-19 07:00:13.289880",
        "proof": 1430,
        "previous_hash":
"21d6a921f991a12c2a765776eb069cb3a1a2f9e4c08499293faad9f6728173cc",
        "transactions": [
            {
                "sender": "Me",
                "receiver": "Saniya",
                "amount": 489.0
            }
        ]
    }
],
"length": 5
}

```

Program #2: Python Program using Flask via VS Code: Flask REST API Blockchain Server

Program:

```

import datetime
import hashlib

```

```

import json
from flask import Flask, jsonify, request

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []
        # Create Genesis Block
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions
        }
        self.transactions = [] # Clear transaction pool after mining
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def create_transaction(self, sender, receiver, amount):
        self.transactions.append({
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        })
        return len(self.chain) + 1

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False

        while check_proof is False:
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:3] == '000':
                check_proof = True
            else:
                new_proof += 1

        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys=True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):

```

```

previous_block = chain[0]
block_index = 1

while block_index < len(chain):
    block = chain[block_index]

    if block['previous_hash'] != self.hash(previous_block):
        return False

    previous_proof = previous_block['proof']
    proof = block['proof']
    hash_operation = hashlib.sha256(
        str(proof**2 - previous_proof**2).encode()
    ).hexdigest()

    if hash_operation[:3] != '000':
        return False

    previous_block = block
    block_index += 1

return True

# -----
# Flask API Routes
# -----
app = Flask(__name__)
blockchain = Blockchain()

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    json_data = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json_data for key in transaction_keys):
        return 'Missing transaction fields', 400

    index = blockchain.create_transaction(
        json_data['sender'],
        json_data['receiver'],
        json_data['amount']
    )
    response = {'message': f'Transaction will be added to Block {index}'}
    return jsonify(response), 201

@app.route('/mine_block', methods=['GET'])
def mine_block():
    if not blockchain.transactions:
        return jsonify({'message': 'No pending transactions to mine!'}), 400

    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)

```



```

block = blockchain.create_block(proof, previous_hash)

response = {
    'message': 'Block mined successfully!',
    'index': block['index'],
    'timestamp': block['timestamp'],
    'proof': block['proof'],
    'previous_hash': block['previous_hash'],
    'transactions': block['transactions']
}

return jsonify(response), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

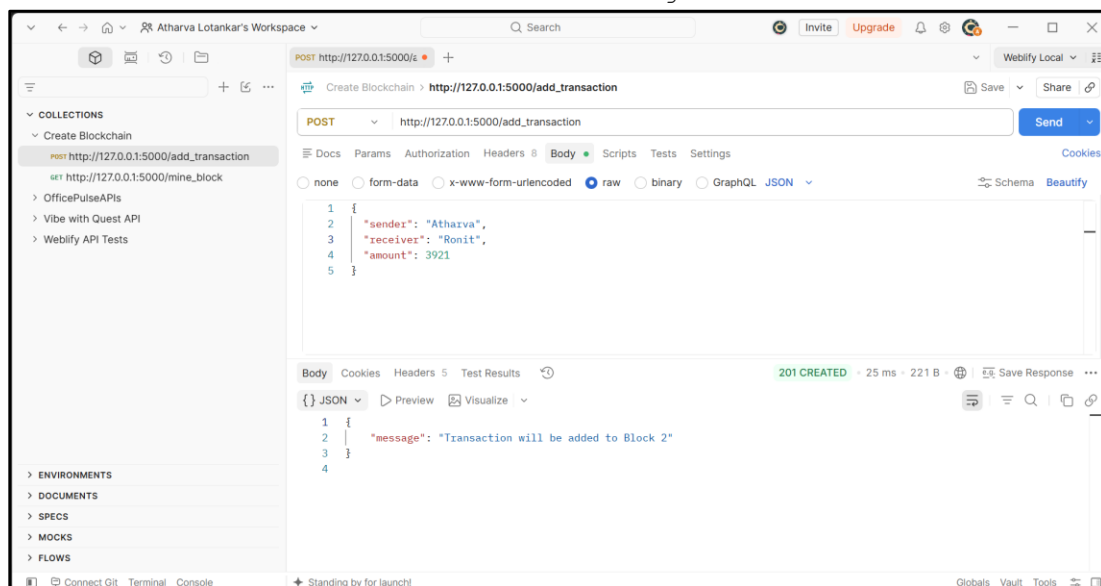
@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(blockchain.chain)
    if valid:
        response = {'message': 'The Blockchain is valid.'}
    else:
        response = {'message': 'The Blockchain is invalid.'}
    return jsonify(response), 200

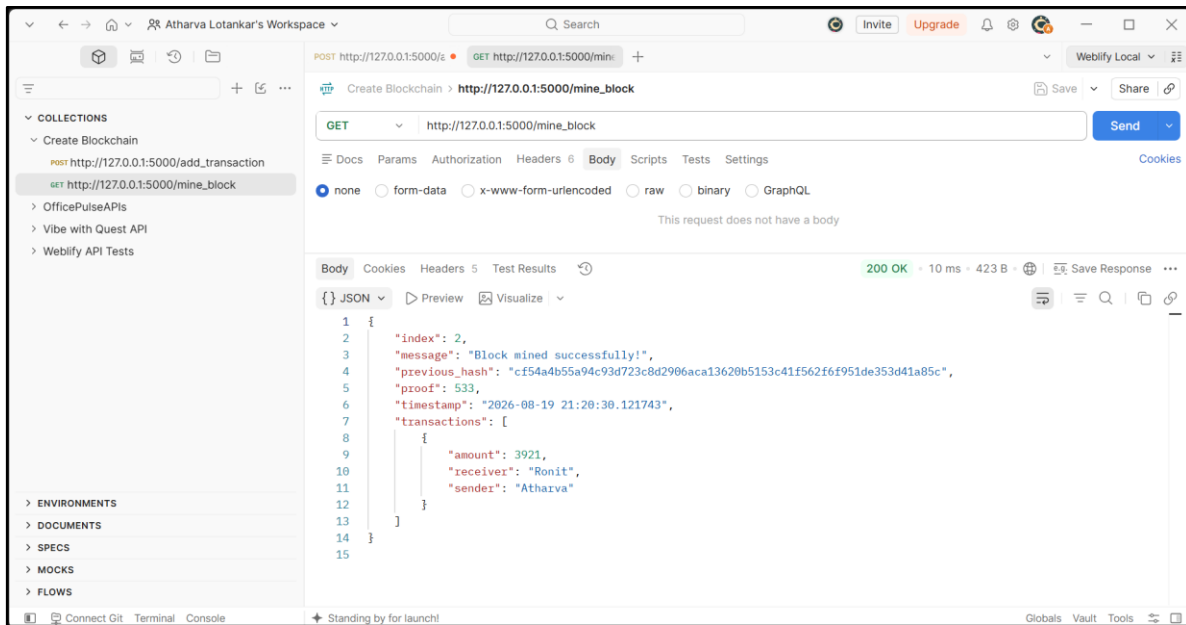
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

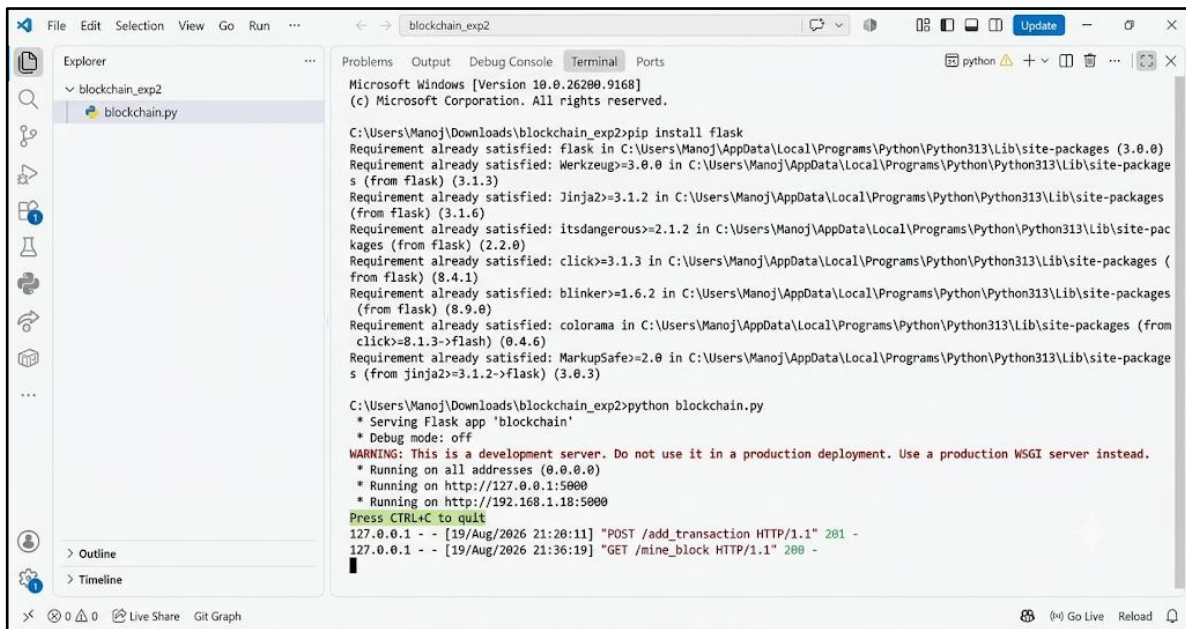
Output:

Postman Testing





VScode Flask Running



Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.